ANÁLISIS DEL RETO

Juan Sebastian Rojas Marin, js.rojasm123@uniandes.edu.co, 202610140

Katy Teran Mercado, k.teranm@uniandes.edu.co, 202614247

Juan Jose Rincon Santos, jj.rincons1@uniandes.edu.co, 202615789

Requerimiento <<6>>

Descripción

```
def req_6(catalog, n, producto, fecha_inicial, fecha_final):
    def criterio(pais_1, pais_2):
        if pais_1["eficiencia"] != pais_2["eficiencia"]:
            return pais_1["eficiencia"] > pais_2["eficiencia"]
        if pais_1["amount_total"] != pais_2["amount_total"]:
            return pais_1["amount_total"] > pais_2["amount_total"]
        return pais_1["Country"] <= pais_2["Country"]

    start_time = get_time()

    pedidos_producto = mlp.get(catalog["OrdenProd"], producto)

    resumen_paises = mlp.new_map(20, 0.5)
    paises_encontrados = lt.new_list()

    total_pedidos = 0
    amount_total = 0.0
    marketing_total = 0.0

    if pedidos_producto is not None:
        for i in range(lt.size(pedidos_producto)):
            pedido = lt.get_element(pedidos_producto, i)
            fecha = pedido["Order_Date"]
            if fecha >= fecha_inicial and fecha <= fecha_final:
                total_pedidos += 1
                amount_total += pedido["Amount"]
                marketing_total += pedido["Marketing_Spend"]
```

```
                pais = pedido["Country"]
                resumen = mlp.get(resumen_paises, pais)
                if resumen is None:
                    resumen = {
                        "Country": pais,
                        "num_pedidos": 0,
                        "amount_total": 0.0,
                        "marketing_total": 0.0,
                        "boxes_total": 0,
                        "suma_precio": 0.0,
                        "precio_promedio": 0.0,
                        "eficiencia": 0.0,
                    }
                    mlp.put(resumen_paises, pais, resumen)
                    lt.add_last(paises_encontrados, resumen)

                resumen["num_pedidos"] += 1
                resumen["amount_total"] += pedido["Amount"]
                resumen["marketing_total"] += pedido["Marketing_Spend"]
                resumen["boxes_total"] += pedido["Boxes_Shipped"]
                resumen["suma_precio"] += pedido["Price_per_Box"]

    total_paises = lt.size(paises_encontrados)

    ranking = lt.new_list()
    for i in range(total_paises):
        resumen = lt.get_element(paises_encontrados, i)
        resumen["precio_promedio"] = resumen["suma_precio"] / resumen["num_pedidos"]
        if resumen["marketing_total"] > 0:
            resumen["eficiencia"] = resumen["amount_total"] / resumen["marketing_total"]
            lt.add_last(ranking, resumen)

    lt.merge_sort(ranking, criterio)
```

```
    top_paises = lt.new_list()
    limite = lt.size(ranking)
    if n < limite:
        limite = n
    for i in range(limite):
        lt.add_last(top_paises, lt.get_element(ranking, i))

    end_time = get_time()
    tiempo = delta_time(start_time, end_time)
    return (tiempo, total_pedidos, total_paises,
            amount_total, marketing_total, top_paises)
```

Identifica los N países con mayor eficiencia de mercadeo —suma (Amount) / suma(Marketing_Spend)— para un producto dentro de un rango de fechas. Usa dos tablas de hash Linear Probing: OrdenProd, construida en la carga, con llave Product y valor la lista de sus pedidos; y una auxiliar creada en la consulta, con llave Country y valor un registro acumulador por país. Se recorre una sola vez la lista del producto, se verifica el rango comparando las fechas como cadenas (el formato YYYY-MM-DD es lexicográficamente ordenable) y se actualiza el país en O(1); un array_list paralelo guarda los países encontrados para no barrer toda la tabla al final. Luego se calculan precio promedio y eficiencia, se descartan los países con Marketing_Spend total en cero, se ordena con Merge Sort y se toman los primeros N, o todos si N los supera.

Entrada	catalog, n, producto, fecha_inicial, fecha_final
Salidas	tiempo, total_pedidos, total_paises, amount_total, marketing_total, top_paises
Implementado (Sí/No)	Implementado por Juan Rojas, Katy Terán y Juan José Rincón

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	**Complejidad**
Buscar el producto en la tabla de hash OrdenProd	O(1) promedio
Crear la tabla de hash auxiliar de países	O(1)
Recorrer los p pedidos del producto y comparar cada fecha contra el rango	O(p)
Por cada uno de los f pedidos dentro del rango: buscar e insertar el país en la tabla auxiliar y acumular sus seis valores	O(1) por pedido entonces es O(f)

Recorrer los c países para calcular el precio promedio y la eficiencia, descartando los de Marketing_Spend igual a cero	O(c)
Ordenar los c países con Merge Sort	O(c log c)
Extraer los primeros N países del ranking (N ≤ c)	O(c)
TOTAL	***O(p + c log c)***

Análisis

El costo está dominado por el recorrido de los pedidos del producto, O(p), porque con solo cinco países el ordenamiento es en la práctica constante. La ganancia depende de la consulta: 70% Dark Bar recorre el 32% del archivo y Milk Classic Bar en seis meses menos del 2%. Se usó Linear Probing en ambas tablas por sus factores de carga bajísimos (0.06 y 0.12), donde el sondeo resuelve casi siempre en la primera posición; la tabla auxiliar de países evita una segunda pasada al agregar todo en el mismo recorrido.